AFRILANG Stdlib

std/c/mlfeature

Français. Bibliothèque AFRILANG ‘std/c/mlfeature‘ (niveau complex, catégorie complex). Elle expose 7 fonction(s) : featSum, featMean, featMin, featMax, featVar, featStd, featNorm.
‘import ”std/c/mlfeature”‘ · ‘use mlfeature‘
- ‘featSum(v list number) → number‘ - ‘featMean(v list number) → number‘ - ‘featMin(v list number) → number‘ - ‘featMax(v list number) → number‘ - ‘featVar(v list number) → number‘ - ‘featStd(v list number) → number‘ - ‘featNorm(v list number) → list number‘ **English.** AFRILANG library ‘std/c/mlfeature‘ (tier complex, category complex). Exports 7 function(s): featSum, featMean, featMin, featMax, featVar, featStd, featNorm.
‘import ”std/c/mlfeature”‘ · ‘use mlfeature‘
- ‘featSum(v list number) → number‘ - ‘featMean(v list number) → number‘ - ‘featMin(v list number) → number‘ - ‘featMax(v list number) → number‘ - ‘featVar(v list number) → number‘ - ‘featStd(v list number) → number‘ - ‘featNorm(v list number) → list number‘

Catégorie / Category	Modules complex (c/)	
Niveau / Tier	complex	June 16, 2026
Fonctions / Functions	7	

Contents

Français

1. Vue d’ensemble

Bibliothèque AFRILANG ‘std/c/mlfeature‘ (niveau complex, catégorie complex). Elle expose 7 fonction(s) : featSum, featMean, featMin, featMax, featVar, featStd, featNorm.

‘import ”std/c/mlfeature”‘ · ‘use mlfeature‘

```
- `featSum(v list number) → number`
- `featMean(v list number) → number`
- `featMin(v list number) → number`
- `featMax(v list number) → number`
- `featVar(v list number) → number`
- `featStd(v list number) → number`
- `featNorm(v list number) → list number`
```

2. Installation & import

Ajoutez le module à votre programme :

```
import "std/c/mlfeature"
use mlfeature
```

```
Niveau : complex · Catégorie : Modules complex (c/)
Domaines avancés – graphes, NLP, réseaux complexes.
```

3. Référence API

```
•
 featSum(v list number) → number•
 featMean(v list number) → number•
 featMin(v list number) → number•
 featMax(v list number) → number•
 featVar(v list number) → number•
 featStd(v list number) → number•
 featNorm(v list number) → list
```

4. Exemples d’utilisation

```
import "std/c/mlfeature"
use mlfeature
```

```
create result = featSum(/* args */)
say result
```

```
create result = featMean(/* args */)
say result
```

```
create result = featMin(/* args */)
say result
```

```
create result = featMax(/* args */)
say result
```

```
create result = featVar(/* args */)
say result
```

```
create result = featStd(/* args */)
say result
```

5. Bonnes pratiques

```
•
 Préférez `import "std/c/mlfeature"` en tête de fichier.•
 Lancez `afrilang check` avant de livrer.•
 Combinez avec les modules stdlib de la même catégorie.•
 Voir /stdlib/ pour le source live et les démos playground.•
 Signalez les problèmes sur GitHub si une export manque.
```

6. Annexe — source

module mlfeature

```
function featSum(v list number) returns number
end
```

```
function featMean(v list number) returns number
end
```

```
function featMin(v list number) returns number
end
```

```
function featMax(v list number) returns number
end
```

```
function featVar(v list number) returns number
end
```

```
function featStd(v list number) returns number
end
```

```
function featNorm(v list number) returns list number
end
end
```

English

1. Overview

AFRILANG library ‘std/c/mlfeature‘ (tier complex, category complex). Exports 7 function(s): featSum, featMean, featMin, featMax, featVar, featStd, featNorm.

‘import ”std/c/mlfeature”‘ · ‘use mlfeature‘

```
- `featSum(v list number) → number`
- `featMean(v list number) → number`
- `featMin(v list number) → number`
- `featMax(v list number) → number`
- `featVar(v list number) → number`
- `featStd(v list number) → number`
- `featNorm(v list number) → list number`
```

2. Installation & import

Add the module to your program:

```
import "std/c/mlfeature"
use mlfeature
```

```
Tier: complex · Category: Complex modules (c/)
Advanced domains — graphs, NLP, complex networks.
```

3. API reference

```
•
 featSum(v list number) → number•
 featMean(v list number) → number•
 featMin(v list number) → number•
 featMax(v list number) → number•
 featVar(v list number) → number•
 featStd(v list number) → number•
 featNorm(v list number) → list
```

4. Usage examples

```
import "std/c/mlfeature"
use mlfeature
```

```
create result = featSum(/* args */)
say result
```

```
create result = featMean(/* args */)
say result
```

```
create result = featMin(/* args */)
say result
```

```
create result = featMax(/* args */)
say result
```

```
create result = featVar(/* args */)
say result
```

```
create result = featStd(/* args */)
say result
```

5. Best practices

```
•
 Prefer `import "std/c/mlfeature"` at the top of your file.•
 Run `afrilang check` before shipping.•
 Combine with related stdlib modules from the same category.•
 See /stdlib/ for the live source and playground demos.•
 Report issues on GitHub if an export is missing or undocumented.
```

6. Appendix — source

module mlfeature

```
function featSum(v list number) returns number
end
```

```
function featMean(v list number) returns number
end
```

```
function featMin(v list number) returns number
end
```

```
function featMax(v list number) returns number
end
```

```
function featVar(v list number) returns number
end
```

```
function featStd(v list number) returns number
end
```

```
function featNorm(v list number) returns list number
end
end
```

Référence rapide / Quick reference

- Import : `import "std/c/mlfeature"`
- Vérification : `afrilang check`
- Documentation : https://afrilang.dev/stdlib/std/c/mlfeature/

Source (extrait)

```
module mlfeature

function featSum(v list number) returns number
end

function featMean(v list number) returns number
```

```
end

function featMin(v list number) returns number
end

function featMax(v list number) returns number
end

function featVar(v list number) returns number
end

function featStd(v list number) returns number
end

function featNorm(v list number) returns list number
end
end
```